

Blocco 5 – Gentle Introduction ai JOIN

Sintassi, esempi guidati e soluzioni

Uso del materiale

Questo handout accompagna la presentazione aggiuntiva del blocco 5. **PDF slide:** `block05_join_cardinalita_introduzione_join.pdf`. Serve per introdurre gradualmente i JOIN, richiamando lo schema normalizzato usato nel blocco 3 per magazzino e vendite.

1. Idea generale

Nel blocco 3 abbiamo separato i dati in più relazioni per evitare duplicazioni e anomalie. Nel blocco 5 impariamo a ricomporre quelle relazioni quando una domanda richiede informazioni distribuite.

- **customers:** una riga per cliente.
- **products:** una riga per prodotto.
- **sales:** una riga per vendita.
- **sale_items:** una riga per prodotto venduto dentro una vendita.

Domanda guida. Per ogni riga vendita vogliamo mostrare data vendita, cliente, città, prodotto, quantità e prezzo applicato.

Problema. Nessuna tabella contiene da sola tutte queste colonne. Il JOIN consente di recuperarle seguendo i collegamenti tra chiavi.

2. Sintassi base

Sintassi.

```
SELECT colonne
FROM tabella_1 AS t1
JOIN tabella_2 AS t2
  ON t2.chiave = t1.chiave_esterna;
```

Ragionamento. FROM sceglie la tabella di partenza. JOIN aggiunge una tabella collegata. ON dichiara la condizione di collegamento. AS assegna alias brevi per rendere leggibile la query.

3. Primo esempio: vendite e clienti

Domanda informale. Mostrare ogni vendita con il nome e la città del cliente.

Specifica corretta.

- granularità: una riga per vendita;
- tabella base: `sales`;
- tabella collegata: `customers`;
- collegamento: `sales.customer_id` punta a `customers.customer_id`.

Soluzione.

```

SELECT
    s.sale_id,
    s.sale_date,
    c.customer_name,
    c.city
FROM sales AS s
JOIN customers AS c
    ON c.customer_id = s.customer_id
ORDER BY s.sale_id;

```

Ragionamento. Ogni vendita ha un cliente obbligatorio. Il risultato resta una riga per vendita perché stiamo aggiungendo una tabella sul lato “uno”.

4. Secondo esempio: vendite e righe vendita

Domanda informale. Mostrare le righe contenute nelle vendite.

Specifica corretta.

- granularità: una riga per riga vendita;
- una vendita può avere molte righe vendita;
- entrando nel lato molti, il numero di righe può aumentare.

Soluzione.

```

SELECT
    s.sale_id,
    s.sale_date,
    si.line_no,
    si.product_id,
    si.quantity
FROM sales AS s
JOIN sale_items AS si
    ON si.sale_id = s.sale_id
ORDER BY s.sale_id, si.line_no;

```

Controllo.

```

SELECT COUNT(*) AS sales_rows
FROM sales;

SELECT COUNT(*) AS joined_rows
FROM sales AS s
JOIN sale_items AS si
    ON si.sale_id = s.sale_id;

```

Ragionamento. Se una vendita contiene più prodotti, dopo il join vedremo più righe per la stessa vendita. Questo comportamento si chiama fan-out.

5. Ricostruire il report iniziale

Problema informale. Viene mostrata una tabella unica con cliente, vendita e prodotto. Bisogna ricostruire lo stesso contenuto partendo dallo schema normalizzato.

Specifica corretta.

- granularità: una riga per riga vendita;
- tabella base logica: `sale_items`;
- aggiungere `sales` per data e cliente;
- aggiungere `customers` per nome e città;
- aggiungere `products` per SKU e nome prodotto.

Soluzione.

```
SELECT
  s.sale_id,
  s.sale_date,
  c.customer_name,
  c.city,
  p.sku,
  p.product_name,
  si.quantity,
  si.unit_price
FROM sale_items AS si
JOIN sales AS s
  ON s.sale_id = si.sale_id
JOIN customers AS c
  ON c.customer_id = s.customer_id
JOIN products AS p
  ON p.product_id = si.product_id
ORDER BY s.sale_id, si.line_no;
```

Ragionamento. Non stiamo tornando a memorizzare una tabella duplicata. Stiamo producendo una vista di lettura a partire da dati mantenuti nel posto giusto.

6. INNER JOIN

Definizione pratica. INNER JOIN conserva solo le righe che trovano una corrispondenza nella tabella collegata.

Quando usarlo.

- il collegamento è obbligatorio;
- ci aspettiamo che ogni riga abbia una riga collegata;
- una mancata corrispondenza sarebbe un'anomalia o un caso da investigare.

Esempio.

```
SELECT si.sale_id, si.line_no, p.product_name
FROM sale_items AS si
JOIN products AS p
  ON p.product_id = si.product_id
ORDER BY si.sale_id, si.line_no;
```

7. LEFT JOIN

Definizione pratica. LEFT JOIN conserva tutte le righe della tabella di sinistra, anche quando non esiste una corrispondenza nella tabella di destra.

Quando usarlo.

- il collegamento è opzionale;
- vogliamo mantenere anche righe senza dettaglio collegato;
- vogliamo cercare casi mancanti, come prodotti mai venduti o clienti senza vendite.

Prodotti mai venduti.

```
SELECT
  p.product_id,
  p.sku,
  p.product_name
FROM products AS p
LEFT JOIN sale_items AS si
  ON si.product_id = p.product_id
WHERE si.sale_id IS NULL
ORDER BY p.product_id;
```

Clienti senza vendite.

```
SELECT
  c.customer_id,
  c.customer_name,
  c.city
FROM customers AS c
LEFT JOIN sales AS s
  ON s.customer_id = c.customer_id
WHERE s.sale_id IS NULL
ORDER BY c.customer_id;
```

8. Collegamenti opzionali: spedizioni

Problema informale. Alcune vendite sono già state spedite, altre no. Vogliamo mostrare le vendite senza spedizione.

Specifica corretta.

- granularità: una riga per vendita non spedita;
- tabella base: `sales`;
- collegamento opzionale: `shipments`;
- cercare i casi in cui la spedizione non esiste.

Soluzione.

```
SELECT
  s.sale_id,
  s.sale_date,
  c.customer_name
FROM sales AS s
JOIN customers AS c
  ON c.customer_id = s.customer_id
LEFT JOIN shipments AS sh
  ON sh.sale_id = s.sale_id
WHERE sh.shipment_id IS NULL
ORDER BY s.sale_id;
```

9. Errore frequente: join senza condizione

Da evitare.

```
SELECT s.sale_id, c.customer_name
FROM sales AS s
JOIN customers AS c
  ON true;
```

Perché è sbagliato. Ogni vendita viene combinata con ogni cliente. Il risultato può avere molte righe, ma quelle righe non rappresentano collegamenti reali.

Forma corretta.

```
SELECT s.sale_id, c.customer_name
FROM sales AS s
JOIN customers AS c
  ON c.customer_id = s.customer_id;
```

10. Query costruita per passaggi

Domanda. Mostrare ogni riga vendita con cliente, prodotto e importo della riga.

Passo 1: granularità. Una riga per riga vendita.

Passo 2: tabella base.

```
SELECT *
FROM sale_items;
```

Passo 3: aggiungere vendita, cliente e prodotto.

```
SELECT
  si.sale_id,
  si.line_no,
  c.customer_name,
  p.product_name,
  si.quantity,
  si.unit_price,
  si.quantity * si.unit_price AS line_amount
FROM sale_items AS si
JOIN sales AS s
  ON s.sale_id = si.sale_id
JOIN customers AS c
  ON c.customer_id = s.customer_id
JOIN products AS p
  ON p.product_id = si.product_id
ORDER BY si.sale_id, si.line_no;
```

11. Esercizi brevi con soluzione

Esercizio A. Mostrare tutte le vendite con cliente e città.

```
SELECT s.sale_id, s.sale_date, c.customer_name, c.city
FROM sales AS s
JOIN customers AS c
  ON c.customer_id = s.customer_id
ORDER BY s.sale_id;
```

Esercizio B. Mostrare tutte le righe vendita con nome prodotto e quantità.

```
SELECT si.sale_id, si.line_no, p.product_name, si.quantity
FROM sale_items AS si
JOIN products AS p
  ON p.product_id = si.product_id
ORDER BY si.sale_id, si.line_no;
```

Esercizio C. Mostrare i clienti che non hanno vendite.

```
SELECT c.customer_id, c.customer_name, c.city
FROM customers AS c
LEFT JOIN sales AS s
  ON s.customer_id = c.customer_id
WHERE s.sale_id IS NULL
ORDER BY c.customer_id;
```

Checklist

- La granularità del risultato è dichiarata?
- La tabella base corrisponde alla granularità desiderata?
- Ogni JOIN ha una condizione ON esplicita?
- Il tipo di join riflette un collegamento obbligatorio o opzionale?
- Il fan-out è previsto e controllato?
- Le colonne sono qualificate con alias leggibili?